LAB 4 – VIBECODING

Project: Reaction Time Tester

Name: Sinchana M.

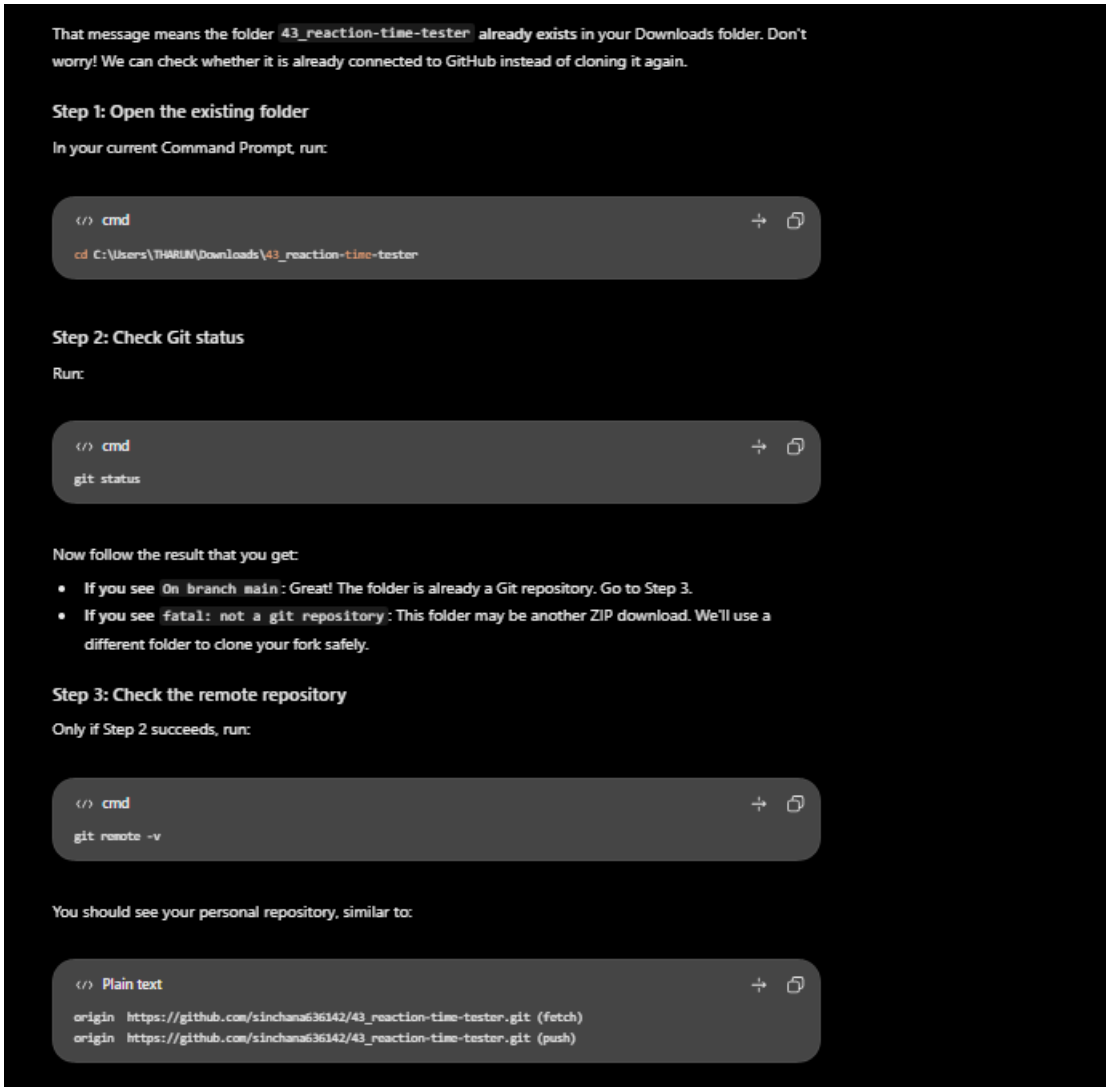
SRN: PES1UG24AM406

AI Tool Used: ChatGPT

GitHub Repository:
https://github.com/sinchana636142/43_reaction-time-tester

Lab Repository:
https://github.com/sinchana636142/PES1UG24AM406_LAB

Objective: To debug the existing Reaction Time Tester application, implement the required features using AI-assisted coding, test the updated application, and maintain the project in GitHub with appropriate commits.



1. The actual issue

Your `main.py` processes input events **before** calling `engine.update()`:

```python
for event in pygame.event.get():
    engine.handle_event(event)

engine.update()
```

This creates a possible timing bug. If the waiting period has elapsed and the screen should turn green, an input event might be handled before the round state changes from `"waiting"` to `"go"`. That input could be incorrectly flagged as a false start.

Your `game/round.py` already measures reaction time from `go_time` correctly, and your engine already excludes false starts from `reaction_times`. We should preserve that logic.

2. Make one small change in `game/game_engine.py`

You don't need to replace all three files. Modify only the `handle_event()` method in `game/game_engine.py`.

Find this section:

```python
        if self.game_over:
            self._handle_game_over_event(event)
            return

        is_click = event.type == pygame.MOUSEBUTTONDOWN
```

Replace it with:

```python
        if self.game_over:
            self._handle_game_over_event(event)
            return

        # Update the waiting/go state before classifying an input.
        # This prevents a reaction after the wait deadline from
        # being incorrectly classified as a false start.
        if self.round.state == "waiting":
            self.round.update()

        is_click = event.type == pygame.MOUSEBUTTONDOWN
```

change the game-over screen, difficulty selection, or sound features.

File 1: `game/round.py`

Open `game/round.py` in VS Code, select all the existing code, and replace it with this:

```python
import random
import pygame


class Round:
    def __init__(self, min_wait_ms=1000, max_wait_ms=3000):
        self.wait_delay_ms = random.randint(min_wait_ms, max_wait_ms)
        self.state = "waiting"

        # Record when the waiting phase begins.
        self.start_time = pygame.time.get_ticks()

        # These are set when the round actually transitions to green.
        self.go_time = None
        self.reaction_ms = None
        self.false_start = False

    def update(self):
        """Transition from waiting to go when the delay expires."""
        if self.state != "waiting":
```

This file retains your existing logic and makes the state transitions explicit. The key calculation is:

```python
self.reaction_ms = now - self.go_time
```

False starts return `None`, not a valid reaction time.

File 2: `game/game_engine.py`

Your engine needs to update the waiting/go state before classifying an input event. Otherwise, a click arriving after the waiting deadline but before the next regular update could be incorrectly classified as a false start.

Replace the entire contents of `game/game_engine.py` with the following code. It preserves your existing game-over screen, difficulty options, and sound effects.

Python

Run

```python
import math
from array import array

import pygame
from .round import Round

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
GRAY = (90, 90, 90)
GREEN = (40, 180, 90)
BLUE = (50, 90, 170)
RED = (190, 55, 55)
DARK = (30, 35, 45)
YELLOW = (245, 205, 80)


class GameEngine:
    def __init__(
        self, width, height, rounds_total=5,
        min_wait_ms=1000, max_wait_ms=3000
```

File 3: `main.py`

Your existing `main.py` can remain unchanged. It already initializes Pygame, processes events, updates the engine, renders the screen, and controls the frame rate.

For clarity, here is the complete version you should have:

Python

Run

```python
# Initialize pygame and start the application.
pygame.init()

# Screen dimensions.
WIDTH, HEIGHT = 600, 400
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Reaction Time Tester - Pygame Version")

# Clock.
clock = pygame.time.Clock()
FPS = 60

# Create the game engine.
engine = GameEngine(WIDTH, HEIGHT)


def main():
    running = True

    while running and not engine.exited:
```

Make this actual change

Open `game/game_engine.py` in VS Code. Find the `handle_event()` method.

Locate this section:

```python
        if not (is_click or is_space):
            return

        if self.round.state not in ("waiting", "go"):
            return
```

Replace it with:

```python
        if not (is_click or is_space):
            return

        # NEW: Update the round before classifying this input.
        # If the waiting deadline has passed, transition to "go" first.
        self.round.update()

        if self.round.state not in ("waiting", "go"):
            return
```

That's the specific change for Task 1. Leave `main.py` and `game/round.py` unchanged for now.

Test the change

1. Save `game_engine.py` with `Ctrl + S`.
2. Run `python main.py`.
3. Click during the grey screen. It should show `FALSE START!`.
4. Wait for green and click. It should show a reaction time in milliseconds.
5. Confirm false starts are not included in the average.

Do this now

1. In the large black editor area, click beside line 1 where it says `Enter file contents here`.

Type this:

Markdown

```
# Lab 4 - Reaction Time Tester

This project implements a Reaction Time Tester using Python and Pygame.

## Files
- main.py
- game/round.py
- game/game_engine.py
- requirements.txt
```

2. Click the green `Commit changes...` **button** at the top right.

3. On the next screen, click **Commit changes** again to save it.

Your folder is now created at:

`PES1UG24AM406_LAB/Lab_4/`

Next, we'll upload the actual Python files into this folder.

Step 1: Verify your GitHub folder

Open `PES1UG24AM406_LAB` → `Lab_4` and check that the structure looks like this:

```
PES1UG24AM406_LAB/
└── Lab_4/
    ├── main.py
    ├── README.md
    ├── requirements.txt
    ├── .gitignore
    └── game/
        ├── round.py
        └── game_engine.py
```

Make sure `round.py` and `game_engine.py` are inside `game/`, not duplicated at the top level.

Step 2: Test your game on your computer

Open the project folder in VS Code and run the game using the terminal:

```powershell
python main.py
```

Yes, you should submit both videos, because your Lab 4 assignment requires a before and an after gameplay video.

What to do

1. **Before video:** Upload the 10-second recording of the original game.
2. **After video:** Upload the 10-second recording of your modified game.
3. **Where:** You can upload both to your `PES1UG24AM406_LAB` repository inside `Lab_4`, or submit them through your college's submission platform if your instructor requires that.

How to upload them to GitHub

1. Open `PES1UG24AM406_LAB` → `Lab_4`.
2. Click **Add file** → **Upload files.**
3. Select both video files from your computer.
4. Enter the commit message `Add Lab 4 before and after videos`.
5. Click **Commit changes.**

Note: GitHub's browser upload limit is 100 MB per file. If either video is too large, use a shareable Google Drive link if your assignment permits it.

Also remember to submit your **AI chat history or complete chat link**, as required by the assignment.

Lab 4 – Submission Summary

The Reaction Time Tester project was updated with AI-assisted coding and debugging.

Project files:

- main.py
- game/round.py
- game/game_engine.py
- requirements.txt
- .gitignore
- README.md

Demo videos:

- before.mp4 — original application
- after.mp4 — updated application

Version control: Project changes were committed and pushed to the personal GitHub repository.

AI assistance: The screenshots in this document record the relevant prompts, responses, debugging exchanges, and implementation guidance used during the project.